DAILY FUSION CODE · WEEK 2

Gyro-radius: your first MATLAB functions

MATLAB | Tue 20 Oct 2026 | CTRL-1.1 | Level 1 | ~10 min

Ions and electrons in a tokamak spiral around the field lines with the **Larmor radius** r_L. You compute it for a 10 keV plasma at $B = 5.3$ T, and on the way write your first MATLAB **functions**: named blocks with inputs, outputs and private variables. Every later CTRL exercise is built from them.

New to MATLAB functions? Read the theory first.

Syntax you'll need

```
function out = name(in1, in2)   % a function: inputs in (), output named out
  out = in1 * in2;              % assign the output before 'end'
end
y = name(3, 4);                 % call it: y is 12
sqrt(x)    abs(q)    x^2        % square root, absolute value, power
fprintf('r = %.3e m\n', r);     % print in scientific notation, then a newline
```

Tasks

Work in `solution.m`; run it by typing `solution` in MATLAB or with `octave-cli solution.m`.

1. **Fill in the two local functions** at the bottom of the file. TODO 1a: `v = thermal_speed(T_eV, m)` returns $v = \sqrt{k_B T/m}$, where $k_B T = e\,T_{\mathrm{eV}}$ in joules. TODO 1b: `r = larmor_radius(v, m, q, B)` returns $r_L = \dfrac{mv}{|q|B}$.

   Hint: each body is one line that assigns the output (`v = ...;`, `r = ...;`). The deuteron lines already call them.

2. **Use them for the electron, then test the walls between functions.** TODO 2a: two lines, the same two calls as for the deuteron but with `m_e`. TODO 2b: uncomment `disp(v)`, run, read the error, comment it out again and finish the last sentence in the file.

   Hint: `thermal_speed` did create a `v`. *Where* does it live?

Check yourself

Deuteron: $r_L \approx 2.73$ mm. Electron: $r_L \approx 0.045$ mm, about 60 times smaller.

Submit

Commit and push `solution.m`. Optional: replace `??` on its first line with your time (e.g. `% time: 10 min`).

THEORY

Scripts, functions and local functions in MATLAB

Script or function?

A **script** is a list of commands in a `.m` file. It runs in the *base workspace*: its variables stay visible afterwards, and it can see (and overwrite) whatever was there. Handy for quick work, risky for bigger programs. A **function** has inputs, outputs and its **own workspace**: its variables vanish when it returns, and only the outputs leave. That isolation is why programs are built from functions.

Several outputs go in brackets, `function [a, b] = name(x, y)`: assign each (`a = ...;`) before `end`, and call with `[p, q] = name(1, 2);`. End lines with `;` or MATLAB prints the result.

Function files and local functions

A **function file** starts with `function` (comments before it are fine). Its first function must have the **same name as the file**; that is the one you call from outside. Functions after it are **local functions**: only code in that file can call them. Put a function in its own file `name.m` when other files need it. MATLAB (since R2016b) also allows local functions at the *end of a script*, but GNU Octave, which checks your code, does not, so here a file with several functions is a function file.

Worked example (different physics, same steps)

The cyclotron frequency is $\omega_c = |q|B/m$, and the frequency in hertz is $f = \omega_c/2\pi$. In a file `demo.m`:

```
function demo()
  w = omega_c(1.602e-19, 1.673e-27, 2.0);   % proton in a 2 T field
  fprintf('f = %.3e Hz\n', w / (2*pi));
end

function w = omega_c(q, m, B)
  w = abs(q) * B / m;
end
```

Typing `demo` prints `f = 3.048e+07 Hz`. Inside `demo`, the names `q`, `m` and `B` do not exist: they belong to `omega_c` and are gone once it returns. Only the *value* of `w` comes back. (Today's physics: $r_L = v_\perp/\omega_c$, and at equal temperature $v \propto 1/\sqrt{m}$, so $r_L \propto \sqrt{m}$.)

Tips and pitfalls

- **Keep file name = first function name**, and run it from its own folder (or add it to the path).
- **Forgot to assign the output?** "Output argument not assigned" (MATLAB) or "value undefined" (Octave).
- **Don't name variables after built-ins** (`sum`, `max`, `i`): the variable hides the function.